

Lab Report No 6
Implementation of Spatial Filtering, Smoothing
and Sharpening
Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

1st Nov, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Example No 1:

Solution:

Brief description (3-5 lines)
Averaging or smoothing filters consist of integration, and they can be used for blurring or noise reduction. Blurring helps to remove small details from an image before further processing like object extraction can be performed on it. Applying a smoothing filter (or any filter for that matter) involves convolving the filter of a specific size with the entire image. The mask moves pixel by pixel while convolving with the values of the pixel intensities
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt # Load the image image = cv2.imread('input_Screenshot 2024-10-26 200104.jpg', 0) #Load in grayscale # Apply Averaging Filter (Simple Blur) average_blur = cv2.blur(image, (5, 5)) #5x5 kernel size # Apply Gaussian Filter gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=1) #5x5 kernel, sigmaX=1 # Display Results plt.figure(figsize=(12, 5)) plt.subplot(1, 3, 1) plt.imshow(image, cmap='gray') plt.title('Original Image') plt.axis('off') plt.subplot(1, 3, 2) plt.imshow(average_blur, cmap='gray') plt.title('Averaging Filter') plt.axis('off') plt.subplot(1, 3, 3) plt.imshow(gaussian_blur, cmap='gray') plt.title('Gaussian Filter')</pre>

```
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Example No 2:

Solution:

Brief description (3-5 lines)

Sharpening filters enhance edges by highlighting areas with sudden intensity changes. Common sharpening filters include:

- **Laplacian Filter:**

Calculates the second derivative of the image to emphasize edges.

- **Unsharp Masking:**

A technique where a blurred version of the image is subtracted from the original.

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale
image_path = ('Screenshot 2024-10-30 124752.jpg' ) # Replace with
your image path if different
image = cv2.imread(image_path, 0)
# Apply Laplacian Filter
laplacian_filter = cv2.Laplacian(image, cv2.CV_64F)
```

```

laplacian_filter = cv2.convertScaleAbs(laplacian_filter) # Convert to
8-bit
# Unsharp Masking
gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=1)
unsharp_image = cv2.addWeighted(image, 1.5, gaussian_blur, -0.5, 0)
# Display Results
plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(laplacian_filter, cmap='gray')
plt.title('Laplacian Filter')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(unsharp_image, cmap='gray')
plt.title('Unsharp Masking')
plt.axis('off')
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Lab Task No 1(a):

Solution:

Brief description (3-5 lines)

Write generic functions for each of the following.

- Takes input mask size and their numeric values from the user and create the mask
 - Take image and padding size as argument and add padding on the image (either zeros or copy neighboring pixel)
 - Take image and filter as argument and apply the filter to the image
 - Take image as input and normalize the image between 0 – 255
- Apply the following Masks the image and display the results

1	1	1
1	1	1
1	1	1



The code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
def create_mask():
    # Get mask size from user
    rows = int(input("Enter the number of rows for the mask: "))
    cols = int(input("Enter the number of columns for the mask: "))
    # Get mask values from user
    values = []
    print("Enter the values for the mask:")
    for _ in range(rows * cols):
        value = int(input())
        values.append(value)
    # Reshape the values into a mask
    mask = np.array(values).reshape((rows, cols))
    return mask
def add_padding(image, padding_size, padding_type='zero'):
```

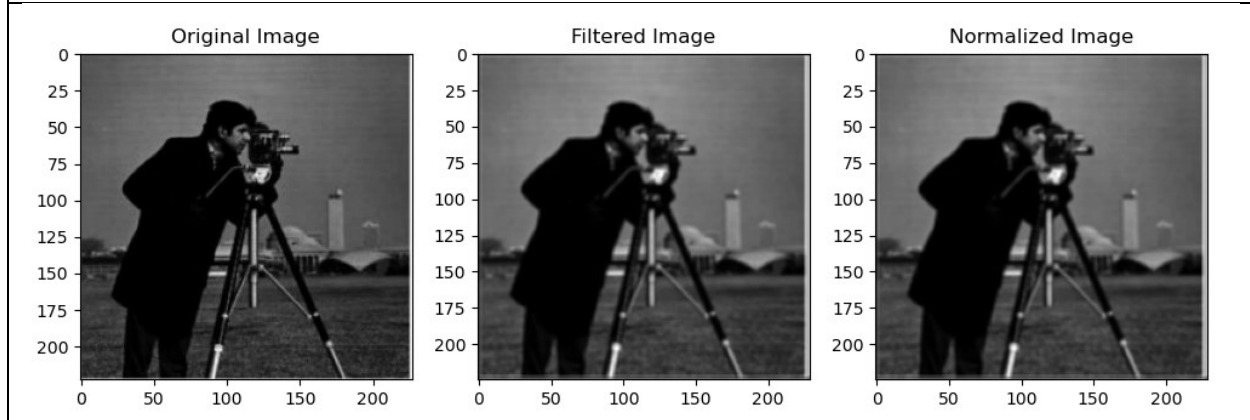
```

    if padding_type == 'zero':
        padded_image = cv2.copyMakeBorder(image, padding_size,
padding_size, padding_size, padding_size,
cv2.BORDER_CONSTANT, value=0)
    elif padding_type == 'replicate':
        padded_image = cv2.copyMakeBorder(image, padding_size,
padding_size, padding_size, padding_size,
cv2.BORDER_REPLICATE)
    return padded_image
def apply_filter(image, mask):
    filtered_image = cv2.filter2D(image, -1, mask)
    return filtered_image
def normalize_image(image):
    normalized_image = cv2.normalize(image, None, 0, 255,
cv2.NORM_MINMAX)
    return normalized_image.astype(np.uint8)
# Load the image in grayscale
image = cv2.imread('Screenshot 2024-11-01 093850.jpg',
cv2.IMREAD_GRAYSCALE)
# Define the 3x3 averaging mask
mask = np.ones((3, 3), np.float32) / 9 # This will create a mask with
all values as 1/9 to average the filter
# Add padding to the image
padding_size = 1 # Set padding size
padded_image = add_padding(image, padding_size,
padding_type='zero')
# Apply the filter to the padded image
filtered_image = apply_filter(padded_image, mask)
# Normalize the filtered image
normalized_image = normalize_image(filtered_image)
# Display the original and processed images
plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 1), plt.imshow(image, cmap='gray'), plt.title('Original
Image')

```

```
plt.subplot(1, 3, 2), plt.imshow(filtered_image, cmap='gray'),
plt.title('Filtered Image')
plt.subplot(1, 3, 3), plt.imshow(normalized_image, cmap='gray'),
plt.title('Normalized Image')
plt.show()
```

The results (Screenshot)



Lab Task No 2:

Solution:

Brief description (3-5 lines)

Apply Horizontal Sobel and Vertical Sobel separately on the given image. Display the results. Then add the images obtained by Horizontal Sobel and Vertical Sobel together and display the resultant image with all the edges.

The code

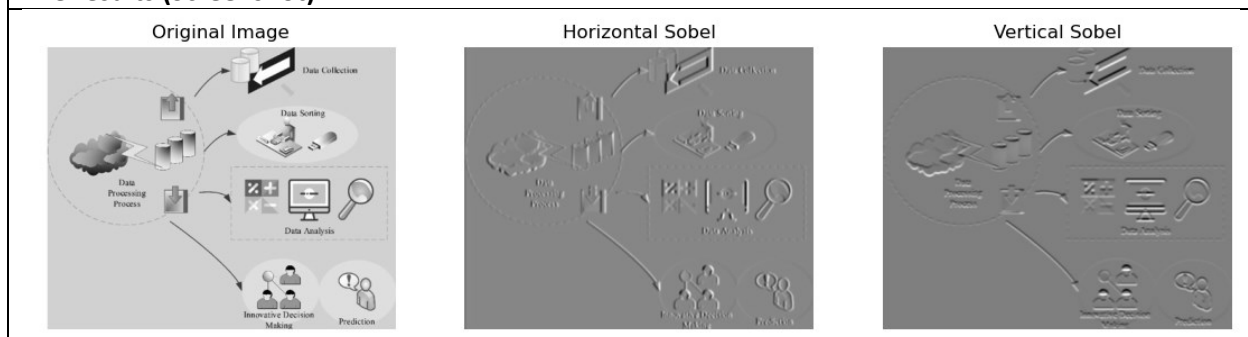
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image
image_path = ('Screenshot 2024-11-01 094346.jpg') # Update this
with the path to your image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
# Apply Horizontal Sobel filter
sobel_horizontal = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=5)
# Apply Vertical Sobel filter
```

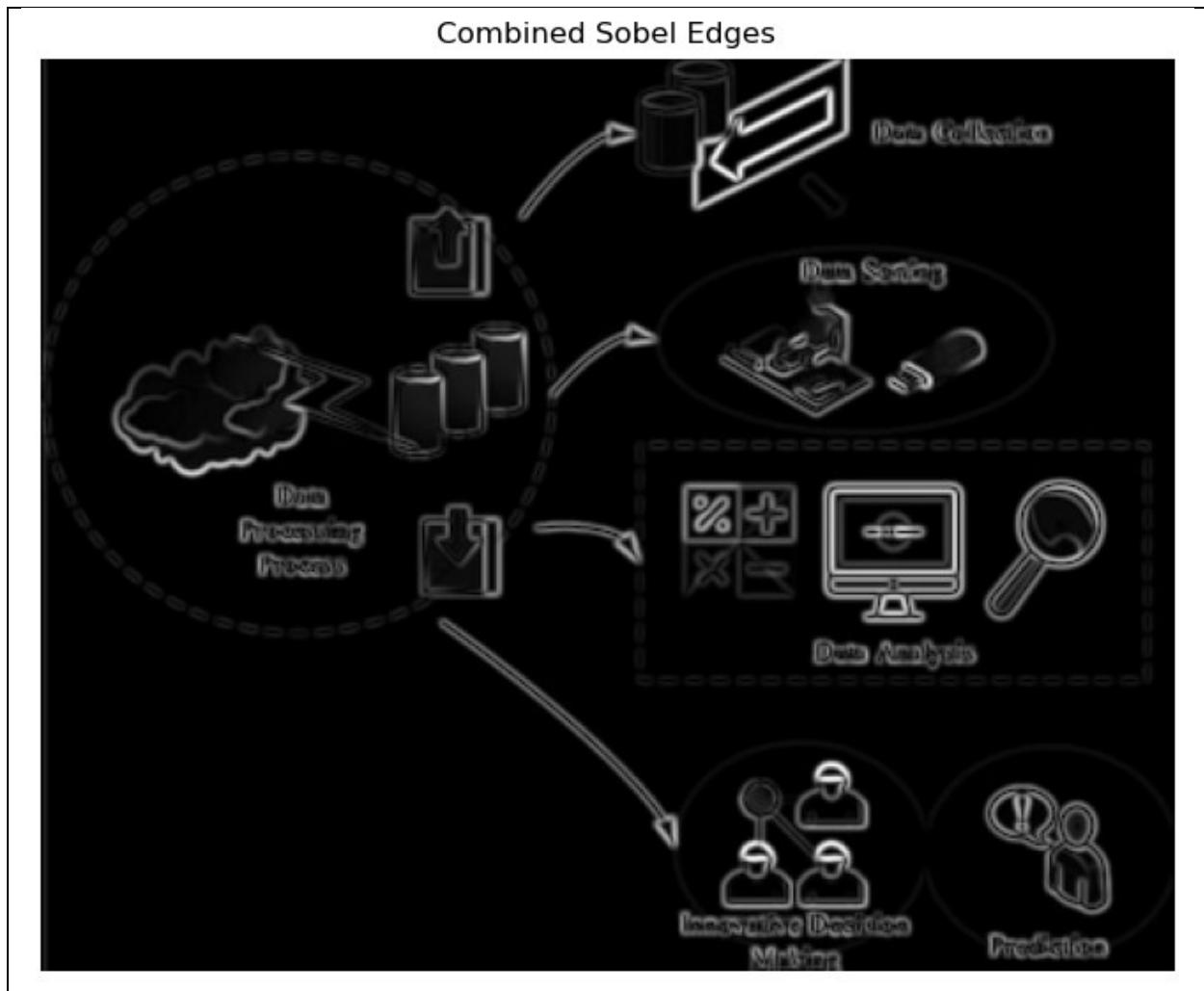
```

sobel_vertical = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=5)
# Combine the results from Horizontal and Vertical Sobel
sobel_combined = cv2.magnitude(sobel_horizontal, sobel_vertical)
# Normalize the combined image to the range of [0, 255]
sobel_combined = cv2.normalize(sobel_combined, None, 0, 255,
cv2.NORM_MINMAX)
# Convert to uint8
sobel_combined = np.uint8(sobel_combined)
# Display the results
plt.figure(figsize=(10, 8))
plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.title('Horizontal Sobel')
plt.imshow(sobel_horizontal, cmap='gray')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.title('Vertical Sobel')
plt.imshow(sobel_vertical, cmap='gray')
plt.axis('off')
plt.figure(figsize=(5, 5))
plt.title('Combined Sobel Edges')
plt.imshow(sobel_combined, cmap='gray')
plt.axis('off')
plt.show()

```

The results (Screenshot)





Lab Task No 3:

Solution:

Brief description (3-5 lines)

Add gaussian noise in our image through skimage.util, random noise.

The code

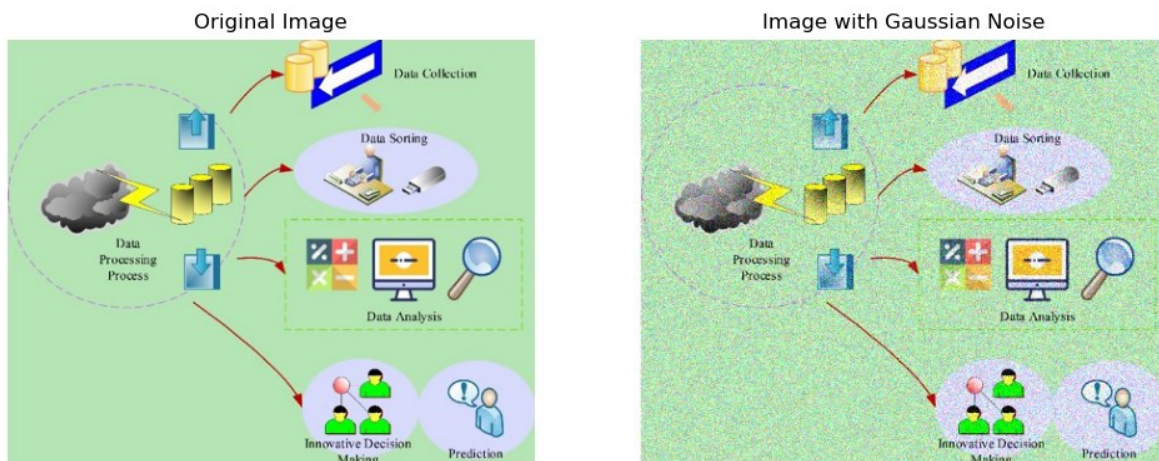
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import io
from skimage.util import random_noise
# Load an image
```

```

image = io.imread('Screenshot 2024-11-01 094346.jpg')
# Add Gaussian noise to the image
noisy_image = random_noise(image, mode='gaussian', var=0.01) #
var controls the amount of noise
# Display the original and noisy images
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image)
plt.axis('off')
plt.subplot(1, 2, 2)
plt.title('Image with Gaussian Noise')
plt.imshow(noisy_image)
plt.axis('off')
plt.show()

```

The results (Screenshot)



Conclusion

In conclusion, the implementation of spatial filtering techniques, including smoothing and sharpening, significantly enhances image quality by reducing noise and improving detail. Smoothing filters effectively blur unwanted artifacts, leading to cleaner images, while sharpening filters accentuate edges and fine features, resulting in greater clarity. The combined application of these techniques enables more accurate image analysis and interpretation in various fields, such as medical imaging, computer vision, and remote sensing. Overall,

spatial filtering serves as a crucial tool in image processing, facilitating better visualization and understanding of complex visual data. Future work may explore advanced filtering methods to further refine and enhance image processing capabilities.